ALGORITHMS · PRINTABLE WORKBOOK

Floors and Ceilings

Big-Omega, Big-Theta and running-time analysis (COMPSCI 311, lecture 3) on paper: the definitions, witnesses you find yourself, loops counted exactly, three loops deep, doing the same job with less work, and 46 questions in six tiers, with an answer key worked out by the same code the website runs.

How to use this packet

1. Work in order: each part is a Learn page with worked examples, a warm-up, then questions.
2. Questions marked **clicker** are the lecture's own.
3. For a witness question, any constants and n_0 that really hold are right. The key shows one and says how to check yours.
4. The threshold is inclusive throughout: "for all $n \geq n_0$", as in the lecture and in CLRS. Questions that name CLRS are adapted from its third edition, which also asks $0 \leq T(n)$ (Part 1 says where that matters).

Part 1	Floors and ceilings	9 questions
Part 2	Count every step	10 questions
Part 3	The triangle	7 questions
Part 4	Polynomial or not	7 questions
Part 5	Three loops deep	6 questions
Part 6	Same answer, less work	7 questions
Summary	One page to keep	
Key	Warm-ups and every question	

PART 1 · LEARN

Floors and ceilings

Big-O only ever gives a ceiling, and a ceiling can be as loose as you like: 10 log n is O(log n), and also O(n), O(n²) and so on. To say how fast something really grows you need a floor as well.

Three definitions, one shape (c, c_1, $c_2 > 0$ and $n_0 \geq 0$ are numbers you choose)

Claim	Means	Picture
T(n) = O(f(n))	$T(n) \leq c \cdot f(n)$ for all $n \geq n_0$	ceiling
T(n) = Ω(f(n))	$T(n) \geq c \cdot f(n)$ for all $n \geq n_0$	floor
T(n) = Θ(f(n))	$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ for all $n \geq n_0$	both: tight

In CLRS (§3.1) the same three read $0 \leq T(n) \leq c \cdot f(n)$, $0 \leq c \cdot f(n) \leq T(n)$ and $0 \leq c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$, with n_0 positive too. The verdicts never change for functions that end up nonnegative, as running times do; the smallest n_0 can. $n - 10 = O(n)$ with $c = 1$ has $n_0 = 0$ here and 10 in CLRS, which also asks $0 \leq n - 10$; and for a T(n) that is never negative, $n_0 = 0$ here is $n_0 = 1$ there.

Worked example: n − 10 = Ω(n) (the slide 8 clicker)

We need $n - 10 \geq cn$. That rearranges to $(1 - c)n \geq 10$, which is possible only if $c < 1$. With $c = 0.5$ it holds from $n = 20$; with $c = 0.99$ from $n = 1000$, where the two sides are exactly equal; with $c = 2$ or 20, never.

n	T(n) = n − 10	0.5·n	T(n) ≥ c·f(n)?
0	-10	0	×
19	9	9.5	×
20	10	10	✓
21	11	10.5	✓
100	90	50	✓

Worked example: 32n² + 17n + 1 = Θ(n²) (slide 11)

Floor: throw the positive terms away. $32n^2 + 17n + 1 \geq 32n^2$ at every n, so $c_1 = 32$ from $n = 0$.

Ceiling: $33n^2$ works once $n^2 \geq 17n + 1$, first true at $n = 18$. Or round every term up: $32n^2 + 17n + 1 \leq 32n^2 + 17n^2 + n^2 = 50n^2$ for $n \geq 1$.

One n_0 for both: the later of the two. $c_1 = 32$, $c_2 = 33$, $n_0 = 18$. It is *not* Θ(n): no c keeps $c \cdot n$ above $32n^2$. And not $\Theta(n^3)$: no c keeps $c \cdot n^3$ below it.

> **Moves that find witnesses.** Ceiling: round every term up to the biggest shape. Floor: drop positive terms; for a negative term, give up part of the constant ($n - 10 \geq \frac{1}{2}n$ once $\frac{1}{2}n \geq 10$). Then solve for n_0, and check the first whole number.

WARM-UP W1 · CEILING OR FLOOR?

Fill in the table for T(n) = 4n + 10 against 5n. From which n is 5n a ceiling? Could 5n ever be a floor?

n	T(n) = 4n + 10	5·n	≤ ?
0			
5			
9			
10			
11			
20			

PART 1 · QUESTIONS

Tier 1: Floors and ceilings

1.1 *Skill: what Big-O does not say*

Clicker · Lecture 3, slide 2

Suppose f is O(g). Which of the following is true?

- ◯ **A.** g is O(f)
- ◯ **B.** g is not O(f)
- ◯ **C.** g may be O(f), depending on the particular functions f and g

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.2 *Skill: choosing a constant for a lower bound*

Clicker · Lecture 3, slide 8

Claim: $n - 10$ is $\Omega(n)$. To prove it we need $n - 10 \geq cn$ for all $n \geq n_0$. What is the largest value of c below for which we can find some n_0 to make this statement true?

- ◯ **A.** $c = 0.5$
- ◯ **B.** $c = 0.99$
- ◯ **C.** $c = 2$
- ◯ **D.** $c = 20$

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.3 *Skill: finding n_0 for a given c*

Take $c = 0.5$. What is the smallest n_0 with $n - 10 \geq 0.5n$ for every $n \geq n_0$?

$n_0 =$ ☐

Working:

1.4 *Skill: the same claim under CLRS's definition*

$n - 10 = O(n)$ with $c = 1$. On this page $n - 10 \leq n$ holds at every n, so the smallest n_0 is 0. CLRS §3.1 defines $O(g(n))$ as the functions $f(n)$ with positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Under that definition, what is the smallest n_0 for $c = 1$?

$n_0 =$ ☐

Working:

1.5 *Skill: producing a lower-bound witness*

Prove $4n + 10 = \Omega(n)$ from the definition: give a $c > 0$ and an n_0 that work.

```
T(n) = 4n + 10    f(n) = n
```

c = ____ **n_0 =** ____

Show it holds at every $n \geq n_0$, not just at a few: the argument, not a table.

1.6 *Skill: a floor and a ceiling at once*

Prove $32n^2 + 17n + 1 = \Theta(n^2)$: give a floor c_1, a ceiling c_2 and one n_0 where both hold.

```
T(n) = 32n² + 17n + 1    f(n) = n²
```

c_1 (floor) = ____ **c_2 (ceiling) =** ____ **n_0 =** ____

Show it holds at every $n \geq n_0$, not just at a few: the argument, not a table.

1.7 *Skill: a floor with a negative term to pay for*

CLRS §3.1 works this one to show that lower-order terms can be ignored. Prove $\frac{1}{2}n^2 - 3n = \Theta(n^2)$: give a floor c_1, a ceiling c_2 and one n_0 where both hold.

```
T(n) = ½n² − 3n    f(n) = n²
```

c_1 (floor) = ____ **c_2 (ceiling) =** ____ **n_0 =** ____

Show it holds at every $n \geq n_0$, not just at a few: the argument, not a table.

1.8 *Skill: there is more than one witness*

Clicker · Lecture 2, slide 8

To show $n^2 + 10^6 n \leq cn^2$ for all $n \geq n_0$, which choices work?

- ◯ **A.** $c = 2, n^0 = 10^6$
- ◯ **B.** $c = 10^6 + 1, n^0 = 1$
- ◯ **C.** Both
- ◯ **D.** Neither

Why not the others? Pick the one you almost chose and say what is wrong with it.

1.9 *Skill: reading O, Ω and Θ off a polynomial*

Let $T(n) = 32n^2 + 17n + 1$. Tick every true statement.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $T(n) = \Theta(n^2)$
- ☐ **B.** $T(n) = \Omega(n)$
- ☐ **C.** $T(n) = O(n^3)$
- ☐ **D.** $T(n) = \Theta(n^3)$
- ☐ **E.** $T(n) = \Theta(n)$
- ☐ **F.** $T(n) = O(n)$

PART 2 · LEARN

Count every step

Asymptotic notation is about a count. Get the count first, exactly if you can, and read the bound off it.

```
Print1(n)
for i = 1 to n do
  print "X"
  for j = 1 to n do
    print "Y"
```

Each of the n rounds prints one X and n Y: **$n(1 + n) = n^2 + n$** characters. At n = 4: XYYYYXYYYYXYYYYXYYYY, 20 characters.

```
Print2(n)
for i = 1 to n do
  print "X"
  if i == 1 then
    for j = 1 to n do
      print "Y"
```

The Y loop runs only when i = 1: **$n + n = 2n$** characters. At n = 4: XYYYYXXX, 8 characters. Nested-looking, but linear.

Many bounds are true; one is tight

$n^2 + n$ is $\Omega(\sqrt{n})$, $\Theta(n^2)$ and $O(n^4)$, all at once. Only $\Theta(n^2)$ pins it down. $2n$ is $\Theta(n)$.

foo and bar (slides 4 and 12)

foo is two nested loops from 1 to n, bar is three. Both are $O(n^3)$, and yet they are not the same: foo is $\Theta(n^2)$, bar $\Theta(n^3)$. A ceiling alone cannot tell two algorithms apart.

WARM-UP W2 · TRACE BY HAND

Write out exactly what Print1 and Print2 print for n = 3, and check the counts against $n^2 + n$ and $2n$.

Print1:

Print2:

PART 2 · QUESTIONS

Tier 2: Count every step

2.1 *Skill: tracing nested loops exactly*

Clicker · Lecture 3, slide 19

```
for i = 1 to n do
  print "X"
  for j = 1 to n do
    print "Y"
```

What is the output of Print1 with n = 4? (ignore spaces)

- ◯ **A.** XYYY XYYY XYYY
- ◯ **B.** XXXX YYYY YYYY YYYY YYYY
- ◯ **C.** XYYYY XYYYY XYYYY XYYYY
- ◯ **D.** XYYYYY XYYYYY XYYYYY XYYYYY

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.2 *Skill: turning a trace into a formula*

Clicker · Lecture 3, slide 20

```
for i = 1 to n do
  print "X"
  for j = 1 to n do
    print "Y"
```

What is the exact number of characters Print1 prints, as a function of n?

- ◯ **A.** n
- ◯ **B.** n^2
- ◯ **C.** $n^2 - n$
- ◯ **D.** $n^2 + n$

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.3 *Skill: several true bounds at once*

Clicker · Lecture 3, slide 21

The running time of Print1 is:

- ◯ **A.** $\Omega(\sqrt{n})$
- ◯ **B.** $\Theta(n^2)$
- ◯ **C.** $O(n^4)$
- ◯ **D.** all of the above

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.4 *Skill: tracing a loop that only runs once*

Clicker · Lecture 3, slide 22

```
for i = 1 to n do
  print "X"
  if i == 1 then
    for j = 1 to n do
      print "Y"
```

What is the output of Print2 with n = 4? (ignore spaces)

- ◯ **A.** XXXX YYYY YYYY YYYY YYYY
- ◯ **B.** XYYYYY XYYYYY XYYYYY
- ◯ **C.** XYYYY X X X
- ◯ **D.** XYYYYYY XYYYYYY XYYYYYY XYYYYYY

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.5 *Skill: counting when the inner loop is guarded*

Clicker · Lecture 3, slide 23

What is the exact number of characters Print2 prints, as a function of n?

- ◯ **A.** n
- ◯ **B.** 2n
- ◯ **C.** $n^2 - n$
- ◯ **D.** n^2

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.6 *Skill: the tight bound on a guarded loop*

Clicker · Lecture 3, slide 24

What is the tight running-time bound of Print2?

- ◯ **A.** $\Theta(\log n)$
- ◯ **B.** $\Theta(n)$
- ◯ **C.** $\Theta(n^2)$
- ◯ **D.** $\Theta(n^3)$

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.7 *Skill: using the formula*

How many characters does Print1 print when n = 10?

Characters = ☐

Working:

2.8 *Skill: why O alone cannot compare two algorithms*

Clicker · Lecture 3, slide 4

foo is two nested loops from 1 to n; bar is three. Fact: both run in $O(n^3)$. Conclusion: foo and bar have the same asymptotic running time. What is wrong?

- ◯ **A.** foo is not $O(n^3)$.
- ◯ **B.** $O(n^3)$ is only a ceiling. foo is really $\Theta(n^2)$ and bar is $\Theta(n^3)$, so they differ.
- ◯ **C.** Nothing: two $O(n^3)$ algorithms are the same speed asymptotically.
- ◯ **D.** bar is not $O(n^3)$.

Why not the others? Pick the one you almost chose and say what is wrong with it.

2.9 *Skill: counting a loop test, which runs once more than its body*

```
1  for j = 2 to A.length
2    key = A[j]
3    // insert A[j] into the sorted A[1 .. j − 1]
4    i = j − 1
5    while i > 0 and A[i] > key
6      A[i + 1] = A[i]
7      i = i − 1
8    A[i + 1] = key
```

CLRS §2.2 counts every line of INSERTION-SORT. On the reversed array 10, 9, …, 1 (n = 10), how many times does the while test on line 5 run?

Times = ☐

Working:

2.10 *Skill: what a bound on "the running time" claims when inputs differ*

INSERTION-SORT takes linear time on a sorted array and quadratic time on a reversed one (CLRS §2.2). Tick every true statement.

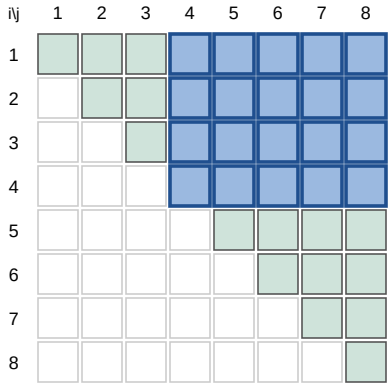
Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** Its running time is $O(n^2)$.
- ☐ **B.** Its running time is $\Omega(n)$.
- ☐ **C.** Its running time is $\Omega(n^2)$.
- ☐ **D.** Its worst-case running time is $\Omega(n^2)$.
- ☐ **E.** Its running time is $\Theta(n^2)$.
- ☐ **F.** Its best-case running time is $\Theta(n)$.

The triangle

```
sum-product
sum = 0
for i = 1 to n do
  for j = i to n do
    sum += A[i]*A[j]
```

Each pair (i, j) the inner line runs on is one cell of an n × n grid, and it runs exactly on the cells with j ≥ i: a triangle, diagonal included.



n = 8. Shaded: every step that runs. Outlined: the easy way's square.

The hard way

Count all of it: n + (n − 1) + … + 1 = n(n + 1)/2. At n = 8: 36. At least $n^2/2$, so $\Omega(n^2)$.

The easy way

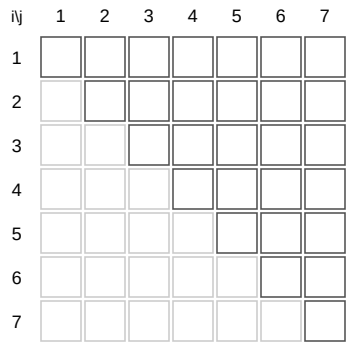
Ignore every step with i > n/2 or j < n/2. What is left is a square of ⌊n/2⌋ × (⌊n/2⌋ + 1) steps, all inside the triangle, so the program does at least that many. At n = 8: 20 ≥ $(n/2)^2$ = 16. A constant times n^2, so $\Omega(n^2)$, without ever finding the exact count.

> **Checking the slide.** "At least $(n/2)^2$" holds for even n. For odd n the square has ⌊n/2⌋(⌊n/2⌋ + 1) steps, a quarter below $(n/2)^2$: at n = 5, 6 against 6.25. The conclusion survives, since it is at least $((n − 1)/2)^2$.

The general moves. Upper bounds: add work that is not there (every loop runs to n). Lower bounds: throw work away (keep a part you can count). Both give $\Theta(n^2)$ here: 20 ≤ 36 ≤ 64.

Additivity, upgraded. If f and g are nonnegative and f = O(g), then f + g = $\Theta(g)$: the floor is free, since f + g ≥ g. So n^2 + 42n + n log n = $\Theta(n^2)$.

WARM-UP W3 · N = 7



Shade the cells sum-product runs on, count them, then outline the easy way's square and count that.

PART 3 · QUESTIONS

Tier 3: The triangle

3.1 *Skill: the hard way: counting exactly*

```
sum = 0
for i = 1 to n do
  for j = i to n do
    sum += A[i]*A[j]
```

How many times does the inner line of sum-product run when n = 10?

Times =

Working:

3.2 *Skill: the easy way: a lower bound by ignoring work*

The easy way keeps only the steps with i ≤ n/2 and j ≥ n/2. When n = 10, how many steps does it keep?

Steps kept =

Working:

3.3 *Skill: why a count of a part is a lower bound on the whole*

Why does counting only the steps in the square prove that sum-product is $\Omega(n^2)$?

- ◯ **A.** Because the square is most of the triangle.
- ◯ **B.** Because $(n/2)^2 = n^2$.
- ◯ **C.** Every step in the square really runs, so the full count is at least the square's count, which is about $n^2/4$.
- ◯ **D.** It proves $O(n^2)$, not $\Omega(n^2)$.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.4 *Skill: why a floor needs a split, not the smallest term*

CLRS Appendix A.2 wants a lower bound on 1 + 2 + … + n, the hard-way count. Its first try bounds every term by the smallest one, 1. What does that give?

- ◯ **A.** n^2, the same as the ceiling.
- ◯ **B.** $n(n + 1)/2$.
- ◯ **C.** A floor of n: true, but only $\Omega(n)$, far below the ceiling n^2, so it cannot show $\Theta(n^2)$.
- ◯ **D.** Nothing: bounding terms from below is not a valid argument.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.5 *Skill: checking a slide's arithmetic*

The slide says the square holds at least $(n/2)^2$ steps. When n = 5 it holds 2 × 3 = 6 steps, and $(5/2)^2 = 6.25$. What follows?

- ◯ **A.** sum-product is not $\Omega(n^2)$.
- ◯ **B.** The line is a little off for odd n, but it still holds at least $((n - 1)/2)^2$, which is $\Omega(n^2)$, so the conclusion stands.
- ◯ **C.** The count must be 7, and 6 is a mistake.
- ◯ **D.** Ω arguments only work for even n.

Why not the others? Pick the one you almost chose and say what is wrong with it.

3.6 *Skill: counting a nested loop that has not been seen before*

```
for i = 1 to n do
  for j = 1 to i do
    for k = 1 to j do
      count += 1
```

How many times does the inner line run when n = 6?

Times = ☐

Working:

3.7 *Skill: additivity, upgraded to Θ*

f and g are nonnegative and f is O(g). Which is true of f + g?

◯ **A.** f + g is Θ(f)

◯ **B.** f + g is O(g) but maybe not Ω(g)

◯ **C.** Nothing, unless f and g are polynomials

◯ **D.** f + g is Θ(g)

Why not the others? Pick the one you almost chose and say what is wrong with it.

PART 4 · LEARN

Polynomial or not

> An algorithm runs in **polynomial time** if its running time is $O(n^d)$ for some constant d.

Running time (slide 16)	Class	Polynomial?	At n = 30, 10^9 steps a second
n	$\Theta(n)$	yes	0.0 µs
4n + 100	$\Theta(n)$	yes	0.2 µs
n log n + 2n + 20	$\Theta(n \log n)$	yes	0.2 µs
$0.01n^2$	$\Theta(n^2)$	yes	0.0 µs
n^2	$\Theta(n^2)$	yes	0.9 µs
$20n^2 + 2n + 3$	$\Theta(n^2)$	yes	18.1 µs
2^n	$\Theta(2^n)$	**no**	1.1 s
3^n	$\Theta(3^n)$	**no**	2 days
n!	$\Theta(n!)$	**no**	8.4e+15 years

Why this is the definition of efficient (slide 17)

- It matches practice: almost all practically efficient algorithms are polynomial.
- It usually separates a clever algorithm from brute force. Stable matching by brute force tries n! matchings (3,628,800 at n = 10); propose-and-reject makes at most $n^2 = 100$ proposals.
- It is refutable: it gives a way to say an algorithm is not efficient, or that no efficient algorithm exists.

Polynomial says nothing about the degree: n^{100} is polynomial. And every exponential passes every polynomial eventually: 2^n overtakes n^{10} between n = 58 and 59, and stays ahead.

PART 4 · QUESTIONS

Tier 4: Polynomial or not

4.1 *Skill: recognising polynomial time*

Tick every running time that is polynomial, meaning $O(n^d)$ for some constant d.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $n \log n + 2n + 20$
- ☐ **B.** $0.01n^2$
- ☐ **C.** $20n^2 + 2n + 3$
- ☐ **D.** 2^n
- ☐ **E.** 3^n
- ☐ **F.** $n!$
- ☐ **G.** n^{100}

4.2 *Skill: why polynomial time is the definition of efficient*

Which are reasons the lecture gives for defining efficient as polynomial time? Tick every one.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** It matches practice: almost all practically efficient algorithms are polynomial.
- ☐ **B.** It usually separates a clever algorithm from brute force.
- ☐ **C.** It is refutable: it lets us say an algorithm is not efficient, or that no efficient algorithm exists.
- ☐ **D.** Every polynomial-time algorithm is fast in practice.

4.3 *Skill: how fast n! grows*

Brute-force stable matching tries every way to pair n students with n colleges. How many is that when n = 10?

Matchings = ☐

Working:

4.4 *Skill: how much a fixed time buys*

CLRS Problem 1-1 asks for the largest problem an algorithm can solve in a given time. At this page's billion steps a second, what is the largest n for which n! steps take at most one second?

n = ____

Working:

4.5 *Skill: where exponential overtakes polynomial*

n^{10} is far bigger than 2^n at n = 10. Which is true?

- ◯ **A.** n^{10} stays ahead, since it starts so far in front.
- ◯ **B.** They take turns being bigger for ever.
- ◯ **C.** 2^n overtakes n^{10} eventually, and stays ahead forever.
- ◯ **D.** They are Θ of each other.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.6 *Skill: which changes to an exponent are constant factors*

CLRS Exercise 3.1-4: is $2^{n+1} = O(2^n)$? Is $2^{2n} = O(2^n)$?

- ◯ **A.** Both: changing the exponent is only a constant factor.
- ◯ **B.** $2^{n+1} = O(2^n)$, but 2^{2n} is not $O(2^n)$.
- ◯ **C.** Neither: anything done to an exponent changes the class.
- ◯ **D.** $2^{2n} = O(2^n)$, but 2^{n+1} is not.

Why not the others? Pick the one you almost chose and say what is wrong with it.

4.7 *Skill: why polynomial time composes*

An $O(n^2)$ algorithm calls an $O(n^3)$ subroutine once for each of its $O(n^2)$ steps, on inputs of size at most n. What is its running time?

- ◯ **A.** $O(n^6)$
- ◯ **B.** $O(n^5)$: still polynomial
- ◯ **C.** $O(2^n)$
- ◯ **D.** $O(n^3)$, since only the largest term counts

Why not the others? Pick the one you almost chose and say what is wrong with it.

PART 5 · LEARN

Three loops deep

The triangle's moves work one dimension up. The count is a sum of sums; the ceiling lets every loop run to n; the floor keeps a **box** of triples where every one really runs, and each side of the box is a constant fraction of n.

```
triangles(n)
for i = 1 to n do
  for j = i + 1 to n do
    for k = j + 1 to n do
      check points i, j, k
```

Exact: $n(n-1)(n-2)/6$. The bands are in order, low, middle, high, so every triple in the box has $i < j < k$, and the loops reach every such triple.

Box at $n = 12$: $i \le n/3$, $n/3 < j \le 2n/3$, $k > 2n/3$: $4 \times 4 \times 4 =$ **64** steps, of 220 in all.

```
deeper(n)
for i = 1 to n do
  for j = i to n do
    for k = 1 to j do
      do one step
```

Exact: $n(n+1)(2n+1)/6$. $i \le n/2 \le j$, so the pair is reached, as in the triangle; and $k \le n/2 \le j$, so k is inside its loop.

Box at $n = 12$: $i \le n/2$, $j \ge n/2$, $k \le n/2$: $6 \times 7 \times 6 =$ **252** steps, of 650 in all.

Choosing a box. Pick a band for each index so that (1) every triple in the box satisfies the loop conditions, and (2) each band has at least a constant fraction of n values. Then the box has at least $c \cdot n^3$ steps, and the program at least that many: $\Omega(n^3)$.

A box that is too thin proves too little. If one band has only 10 values, the box has about $10 \cdot n^2$ steps: a true floor, but only $\Omega(n^2)$.

WARM-UP W5 · DEEPER AT N = 5

Count deeper's steps at $n = 5$ exactly, then count its box ($i \le n/2$, $j \ge n/2$, $k \le n/2$), and check the box fits under the count and the count under n^3.

Tier 5: Three loops deep

5.1 *Skill: counting a triple loop whose ranges depend on the loops outside*

```
for i = 1 to n do
  for j = i + 1 to n do
    for k = j + 1 to n do
      check points i, j, k
```

How many times does the inner line run when n = 8?

Times = ☐

Working:

5.2 *Skill: summing the cells of the (i, j) grid*

```
for i = 1 to n do
  for j = i to n do
    for k = 1 to j do
      do one step
```

How many times does the inner line run when n = 6?

Times = ☐

Working:

5.3 *Skill: the size of a box of triples*

```
for i = 1 to n do
  for j = i + 1 to n do
    for k = j + 1 to n do
      check points i, j, k
```

For triangles, the floor keeps only the triples with i ≤ n/3, n/3 < j ≤ 2n/3 and k > 2n/3. How many triples does it keep when n = 9?

Triples kept = ☐

Working:

5.4 *Skill: choosing a box where every triple really runs*

```
for i = 1 to n do
  for j = i + 1 to n do
    for k = j + 1 to n do
      check points i, j, k
```

To show triangles is Ω(n³), you count a box of triples and claim the program does at least that much work. Which box makes the argument valid?

- ◯ **A.** i ≤ n/2, j ≤ n/2, k ≥ n/2
- ◯ **B.** i, j and k all ≤ n/3
- ◯ **C.** i ≤ n/3, n/3 < j ≤ 2n/3, k > 2n/3
- ◯ **D.** Every i, j, k from 1 to n: n³ triples

Why not the others? Pick the one you almost chose and say what is wrong with it.

5.5 *Skill: O, Ω and Θ for a triple loop*

```
for i = 1 to n do
  for j = i to n do
    for k = 1 to j do
      do one step
```

deeper runs its inner line n(n + 1)(2n + 1)/6 times. Tick every true statement.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** $\Theta(n^3)$
- ☐ **B.** $O(n^4)$
- ☐ **C.** $\Omega(n^2)$
- ☐ **D.** $O(n^2)$
- ☐ **E.** $\Theta(n^2)$
- ☐ **F.** $\Omega(n^4)$

5.6 *Skill: a Θ witness for a triple loop*

Prove $n(n-1)(n-2)/6 = \Theta(n^3)$: give a floor c_1, a ceiling c_2 and one n_0 where both hold.

$T(n) = 1/6n^3 - \frac{1}{2}n^2 + 1/3n \quad f(n) = n^3$

c_1 (floor) = ☐ **c_2 (ceiling) =** ☐ **n_0 =** ☐

Show it holds at every $n \geq n_0$, not just at a few: the argument, not a table.

PART 6 · LEARN

Same answer, less work

```
powers, from scratch
for k = 1 to n do
  p = 1
  for t = 1 to k do
    p = p · x
  P[k] = p
```

n(n + 1)/2 multiplications: 55 at n = 10. Θ(n²).

```
powers, reusing the last one
p = 1
for k = 1 to n do
  p = p · x
  P[k] = p
```

n multiplications: 10 at n = 10. Θ(n).

Same table, entry for entry. The slow one rebuilt x^{k-1} inside every x^k; the fast one kept it. **Strictly faster** means the ratio goes to 0: here n / (n(n + 1)/2) = 2/(n + 1) → 0. A version that did half the work would be faster, but not strictly: the ratio would be ½.

The output is a floor. Whatever the algorithm, it must write its answer. The powers table has n entries, so Ω(n): the fast version is as good as it gets. A times table has n² entries (100 at n = 10), so no algorithm fills it in less than Ω(n²).

WARM-UP W6 · POWERS AT N = 5

Count the multiplications each version makes at n = 5, and write out the table they both produce (x = 3).

PART 6 · QUESTIONS

Tier 6: Same answer, less work

6.1 *Skill: counting the work of recomputing from scratch*

```
for k = 1 to n do
  p = 1
  for t = 1 to k do
    p = p · x
  P[k] = p
```

This program fills P[k] = x^k for k = 1 to n. How many multiplications does it do when n = 10?

Multiplications =

Working:

6.2 *Skill: how the gap between two algorithms grows*

At n = 199, how many times as many multiplications does the from-scratch program do as the one that reuses the last power?

Times as many = ☐

Working:

6.3 *Skill: finding the repeated work*

```
for k = 1 to n do
  p = 1
  for t = 1 to k do
    p = p · x
  P[k] = p
```

Both programs write the same table P[1..n]. Why does the from-scratch one do $\Theta(n^2)$ work?

- ◯ **A.** Its multiplications are slower.
- ◯ **B.** It writes more entries.
- ◯ **C.** To get x^k it recomputes x^{k-1} on the way, although it wrote that into P[k − 1] a moment ago.
- ◯ **D.** It has two nested loops, and two nested loops are always $\Theta(n^2)$.

Why not the others? Pick the one you almost chose and say what is wrong with it.

6.4 *Skill: reuse in a polynomial*

```
y = 0
for i = n downto 0 do
  y = a[i] + x · y
```

CLRS Problem 2-3 evaluates $a_0 + a_1x + \ldots + a_nx^n$ with Horner's rule. How many multiplications does it do when n = 10?

Multiplications = ☐

Working:

6.5 *Skill: what strictly faster means*

Algorithm A takes f(n) steps and algorithm B takes g(n). Which fact shows B is asymptotically faster, not just faster by a constant factor?

- ◯ **A.** $g(n)/f(n) \to 0$ as n grows.
- ◯ **B.** $g(n) \le f(n)$ for every n.
- ◯ **C.** $g = O(f)$.
- ◯ **D.** g has a smaller leading coefficient than f.

Why not the others? Pick the one you almost chose and say what is wrong with it.

6.6 *Skill: the output size as a lower bound*

Some algorithm fills an n × n table with T[i, j] = i · j. Before seeing its code, what can you say about its running time?

- ◯ **A.** $\Omega(n)$: it has to look at n.
- ◯ **B.** $\Omega(n^2)$: it writes n^2 entries, and each write is at least one step.
- ◯ **C.** It could be $O(n \log n)$ with a clever enough trick.
- ◯ **D.** Nothing: a lower bound needs the code.

Why not the others? Pick the one you almost chose and say what is wrong with it.

6.7 *Skill: when the output floor is strong and when it is weak*

Tick every task where the size of the output alone forces $\Omega(n^2)$ time.

Tick every one that is true. Partly right counts as wrong.

- ☐ **A.** Write the n × n times table.
- ☐ **B.** Given n numbers A[1..n], write A[i] + A[j] into an n × n table for every i and j.
- ☐ **C.** Write every product i · j · k for i, j, k from 1 to n.
- ☐ **D.** Write the powers x^1 to x^n.
- ☐ **E.** Say whether n numbers contain a repeat.
- ☐ **F.** Sort n numbers.

KEEP THIS PAGE

One-page summary

Definitions

- O: $T \le c \cdot f$ from n_0 on. A ceiling, and it can be loose.
- Ω: $T \ge c \cdot f$ from n_0 on. A floor.
- Θ: both, $c_1 \cdot f \le T \le c_2 \cdot f$, one n_0 (the later one). Tight.
- $f = O(g)$ says nothing about whether $g = O(f)$.
- CLRS also asks $0 \le T$ (for O), $0 \le c \cdot f$ (for Ω) and $n_0 > 0$: same verdicts, but the smallest n_0 can move.

Finding witnesses

- Ceiling: round every term up to the biggest shape.
- Floor: drop positive terms; pay for negative ones with part of the constant.
- Solve for n_0, then check the first whole number (≥ allows equality).
- For Ω, c must be below the leading coefficient; for O, above it (or equal, if the rest is ≤ 0).

Counting

- Count exactly first; the bound comes from the count.
- Add per round, don't multiply by nesting depth (Print2 is 2n).
- Upper bound: add work that is not there. Lower bound: throw work away.
- $1 + 2 + \ldots + n = n(n + 1)/2 = \Theta(n^2)$.
- If $f = O(g)$ and both ≥ 0: $f + g = \Theta(g)$.

Polynomial time

- $O(n^d)$ for some constant d.
- log < polynomial < exponential < factorial.
- Brute force is often n! or 2^n; the clever algorithm is polynomial.

ANSWER KEY

Answer key: warm-ups

Try everything before you look.

W1 · 4n + 10 against 5n

n	T(n) = 4n + 10	5·n	T(n) ≤ c·f(n)?
0	10	0	×
5	30	25	×
9	46	45	×
10	50	50	✓
11	54	55	✓
20	90	100	✓

c = 5 is a **ceiling** from n = 10: 4n + 10 ≤ 5n once n ≥ 10. As a floor it fails from n = 11 on, for good: 5 is above the leading coefficient 4. So 4n + 10 = O(n) with c = 5, n_0 = 10, and 4n + 10 = Ω(n) needs c ≤ 4.

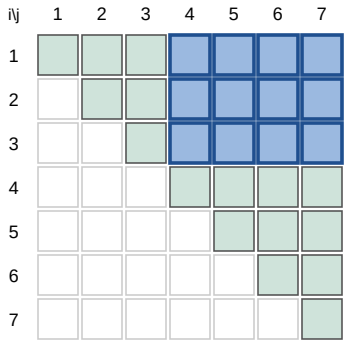
W2 · Print1 and Print2 at n = 3

Print1: XYYYXYYYXYYY: 12 characters, and $n^2 + n = 9 + 3 = 12$.

Print2: XYYYXX: 6 characters, and 2n = 6.

W3 · sum-product at n = 7

All of it: 7 + 6 + … + 1 = 7·8/2 = **28**. The square: i ≤ 3.5 and j ≥ 3.5, so 3 rows × 4 columns = **12**. The slide's $(n/2)^2$ would be 12.25: for odd n the square is a quarter short of it, but $12 \geq ((7 - 1)/2)^2 = 9$, which is all $\Omega(n^2)$ needs.



W5 · deeper at n = 5

Exact: 55 ($1^2 + 2^2 + \ldots + 5^2$). Box: i ≤ 2, j ≥ 3, k ≤ 2: 2 × 3 × 2 = 12. Ceiling: $5^3 = 125$. So 12 ≤ 55 ≤ 125.

W6 · powers at n = 5

From scratch: 1 + 2 + 3 + 4 + 5 = 15 multiplications. Reusing: 5. Both write 3, 9, 27, 81, 243.

Answer key: questions

1.1

Answer: **C**

- **A** × Not always: n is $O(n^2)$, but n^2 is not $O(n)$.
- **B** × Not always: 2n is $O(n)$, and n is $O(2n)$ as well.
- **C** ✓ Both happen. When it does, f and g are Θ of each other; when it does not, f grows strictly slower.

Takeaway. O is a ceiling, like ≤. Knowing $a \le b$ tells you nothing about whether $b \le a$. That gap is exactly why the lecture brings in Ω and Θ. CLRS Problem 3-4(a) asks the same thing, as a conjecture to disprove.

1.2

Answer: **B**

- **A** × It works, from n = 20, but it is not the largest that does.
- **B** ✓ $n - 10 \ge 0.99n$ means $0.01n \ge 10$, which is $n \ge 1000$. Late, but it holds from there on.
- **C** × 2n is bigger than $n - 10$ at every $n \ge 0$. No n_0 helps.
- **D** × Even further out of reach than c = 2.

Takeaway. Every c below 1 works and every c from 1 up fails, however long you wait. For Ω the constant has to fit under the leading coefficient, just as for O it has to clear it.

1.3

Answer: **$n_0 = 20$**

$n - 10 \ge 0.5n$ means $0.5n \ge 10$, so $n \ge 20$. At n = 20 both sides are exactly 10, and "≥" allows equality.

- **If you wrote 21:** At n = 20 both sides are 10, and the definition says ≥, so 20 already counts.
- **If you wrote 10:** That is where $n - 10$ stops being negative, not where it reaches 0.5n.
- **If you wrote 19:** At n = 19 the left side is 9 and the right side is 9.5, so it still fails.

Takeaway. Solving the inequality for n gives n_0 directly. Every witness on this page is found that way, and then checked at whole numbers.

1.4

Answer: **$n_0 = 10$**

CLRS also asks $0 \le n - 10$, which first holds at n = 10, where $n - 10$ is exactly 0. From there both halves hold, and before it the left half fails.

- **If you wrote 0:** That is this page's answer, where only $n - 10 \le n$ is asked. CLRS asks $0 \le n - 10$ as well, and $n - 10$ is negative below 10.
- **If you wrote 1:** CLRS does want n_0 positive, but at n = 1 the function is −9, below the 0 its definition asks for.
- **If you wrote 11:** At n = 10, $n - 10$ is 0, and $0 \le 0$ holds, so 10 already counts.

Takeaway. The two definitions agree on which functions are O(n), since CLRS assumes every function inside the notation is eventually nonnegative, as every running time is. What moves is the smallest n_0: here from 0 to 10, and, for a function that is never

negative, from 0 to 1 whenever this page's answer is 0. CLRS's own witness for $an + b = O(n^2)$, $c = a + |b|$ and $n_0 = \max(1, -b/a)$, carries both conditions: $-b/a$ is where $an + b$ stops being negative.

1.5

Any constants and n_0 that really hold are right. One that does: **$c = 4$, $n_0 = 0$** (checked exactly at every whole n from n_0 on).

$c = 4$ and $n_0 = 0$: $4n + 10 \geq 4n$ at every n, since $10 \geq 0$. Any $c \leq 4$ works from $n = 0$, and c just above 4 works for a while and then fails. (CLRS wants n_0 positive, so there it would be $n_0 = 1$.)

Takeaway. For a floor, the easy move is to drop terms: $4n + 10 \geq 4n$ because the dropped term is positive. For a ceiling, the move is to round up.

1.6

Any constants and n_0 that really hold are right. One that does: **$c_1 = 32$, $c_2 = 33$, $n_0 = 18$** (checked exactly at every whole n from n_0 on).

$c_1 = 32$ is a floor from $n = 0$, since $17n + 1 \geq 0$. $c_2 = 33$ needs $n^2 \geq 17n + 1$, which first holds at $n = 18$. So $c_1 = 32$, $c_2 = 33$, $n_0 = 18$. $c_2 = 50$ works from $n = 1$.

Takeaway. Θ is two witnesses stapled together. Find each side on its own, then use the larger n_0 for both.

1.7

Any constants and n_0 that really hold are right. One that does: **$c_1 = 1/14$, $c_2 = 1/2$, $n_0 = 7$** (checked exactly at every whole n from n_0 on).

Divide by n^2: $c_1 \leq \frac{1}{2} - 3/n \leq c_2$. The right side holds at every $n \geq 1$ with $c_2 = \frac{1}{2}$, since $3/n$ is positive. With $c_1 = 1/14$ the left side needs $3/n \leq \frac{1}{2} - 1/14 = 3/7$, so $n \geq 7$, where both sides are exactly 3.5. Those are CLRS's own constants: $c_1 = 1/14$, $c_2 = \frac{1}{2}$, $n_0 = 7$.

Takeaway. The $-3n$ costs part of the floor's constant, just as -10 does against n. Any c_1 below $\frac{1}{2}$ works once n is large enough; $c_1 = \frac{1}{2}$ itself never does.

1.8

Answer: **C**

- **A** × Works, but so does B.
- **B** × Works, but so does A.
- **C** ✓ A: from $n = 10^6$, $10^6 n \leq n^2$. B: from $n = 1$, $10^6 n \leq 10^6 n^2$. Different pairs, same claim.
- **D** × Both work, each for its own reason.

Takeaway. A witness is any pair that works. Trading a bigger c for a smaller n_0 is always allowed, and a proof only needs one pair.

1.9

True: **A, B, C** · False: D, E, F

- **A** ✓ Floor $32n^2$, ceiling $33n^2$ from $n = 18$.
- **B** ✓ $T(n) \geq n$ at every $n \geq 0$. A floor can be as low as you like.
- **C** ✓ A ceiling can be as high as you like.
- **D** × The floor fails: $c \cdot n^3$ passes $32n^2$ for every $c > 0$ eventually.
- **E** × The ceiling fails: $32n^2$ passes $c \cdot n$ for every c.

- **F** × Same ceiling, same failure.

Takeaway. $\Theta(n^2)$ is the one that says the most. Each of the other true lines is a weaker claim.

2.1

Answer: **C**

- **A** × Three rounds of three Y: that is n = 3, or a loop that stops one early.
- **B** × That prints all the X first. The X is inside the outer loop, before each round of Y.
- **C** ✓ Each of the 4 rounds prints one X and then 4 Y.
- **D** × Five Y per round: the inner loop runs n = 4 times, not 5.

Takeaway. Trace the first two rounds by hand and the pattern writes itself.

2.2

Answer: **D**

- **A** × That is the X alone.
- **B** × That is the Y alone.
- **C** × Nothing is subtracted: the X are printed as well as the Y.
- **D** ✓ n rounds, each printing 1 X and n Y: $n(1 + n)$.

Takeaway. Count per round, then multiply by the number of rounds. Exact counts come first; the bound is read off them.

2.3

Answer: **D**

- **A** × True, since $n^2 + n \geq \sqrt{n}$. But it is not the only true one.
- **B** × True, and the most informative. But it is not the only true one.
- **C** × True, as a loose ceiling. But it is not the only true one.
- **D** ✓ Each is a correct statement. Only $\Theta(n^2)$ is tight.

Takeaway. Being true and being tight are different things. Exams often ask for the tight one.

2.4

Answer: **C**

- **A** × All the X come first here, and far too many Y.
- **B** × That ignores the if: the Y loop only runs when i = 1.
- **C** ✓ Round 1 prints X and 4 Y. Rounds 2 to 4 print just X.
- **D** × That runs the Y loop every round, with too many Y as well.

Takeaway. A loop nested inside another loop does not mean multiplying. Here the inner loop runs in exactly one round.

2.5

Answer: **B**

- **A** × That is the X alone.
- **B** ✓ n X, plus n Y from the one round where i = 1.

- **C** × That treats the inner loop as running every round.
- **D** × Only if the inner loop ran every round.

Takeaway. Add up what each round prints; do not multiply by the loop depth.

2.6

Answer: **B**

- **A** × Nothing here halves anything. 2n grows faster than log n.
- **B** ✓ 2n is Θ(n): floor n, ceiling 2n.
- **C** × The nesting suggests it, but the inner loop runs once, not n times.
- **D** × Far above the 2n it really takes.

Takeaway. Nested-looking code can be linear. Count, then bound.

2.7

Answer: **Characters = 110**

$n^2 + n$ at $n = 10$: $100 + 10 = 110$.

- **If you wrote 100:** That is only the Y. Each of the 10 rounds also prints one X.
- **If you wrote 90:** $n^2 - n$. The X are added, not taken away.
- **If you wrote 20:** That is Print2's count, 2n.

Takeaway. The page's tracer counts every character as it prints. Try n = 10 there and compare.

2.8

Answer: **B**

- **A** × It is: $n^2 \le n^3$ for every $n \ge 1$.
- **B** ✓ Both facts are true and the conclusion does not follow from them. Comparing needs tight bounds.
- **C** × $O(n^3)$ includes everything from constant time up to n^3.
- **D** × It is exactly n^3 steps.

Takeaway. To say two running times are the same, show both are Θ of the same thing.

2.9

Answer: **Times = 54**

Each A[j] is smaller than everything before it, so the loop shifts all $j - 1$ of them, and the test runs once more to stop at $i = 0$: $t_j = j$. Adding j for j = 2 to 10 gives $10 \cdot 11/2 - 1 = 54$.

- **If you wrote 45:** That is line 6, the loop body: the sum of $t_j - 1$. The test runs once more than the body for each j, the time it fails.
- **If you wrote 55:** That is $1 + 2 + \ldots + 10$. j starts at 2: A[1] on its own is already sorted.
- **If you wrote 9:** That is the best case, an array already sorted, where the test fails at once for every j: $t_j = 1$.

Takeaway. CLRS counts the loop tests as well as the bodies and gives each line its own cost c_i. That moves the constants, never the order of growth: the worst case comes out as $an^2 + bn + c$, so $\Theta(n^2)$, and the best case as $an + b$, so $\Theta(n)$.

2.10

True: **A, B, D, F** · False: C, E

- **A** ✓ The worst case is $\Theta(n^2)$, and a ceiling on the worst case is a ceiling on every input.
- **B** ✓ Every input of size n costs at least the best case, which is linear.
- **C** × A sorted array takes only linear time, so no $c \cdot n^2$ stays under the running time on every input.
- **D** ✓ The reversed array makes about $n^2/2$ shifts.
- **E** × That needs $\Omega(n^2)$ on every input as well, and sorted arrays break it.
- **F** ✓ $t_j = 1$ for every j: an + b.

Takeaway. A bound on "the running time", with no modifier, has to hold for every input of each size: O is then a ceiling on the worst case and Ω a floor on the best case. CLRS §3.1 says so on pp. 48 to 49, and Exercise 3.1-6 makes it exact: the running time is $\Theta(g(n))$ when the worst case is $O(g(n))$ and the best case is $\Omega(g(n))$, and only then.

3.1

Answer: **Times = 55**

For i = 1 the inner loop runs 10 times, for i = 2 nine times, down to once for i = 10: $10 + 9 + \ldots + 1 = 10 \cdot 11/2 = 55$.

- **If you wrote 100:** That is n^2, as if j started at 1. It starts at i.
- **If you wrote 45:** That is $n(n-1)/2$, as if j started at i + 1. When j = i the line runs too.
- **If you wrote 50:** That is $n^2/2$. The exact count has an extra n/2 from the diagonal, where i = j.

Takeaway. The triangle including its diagonal: $n(n+1)/2$, which is $\Theta(n^2)$.

3.2

Answer: **Steps kept = 30**

i from 1 to 5, and j from 5 to 10: $5 \times 6 = 30$. At least $(n/2)^2 = 25$, which is all the argument needs.

- **If you wrote 25:** That is the $(n/2)^2$ the slide quotes. The exact count is a little more, since j = n/2 is kept too.
- **If you wrote 55:** That is every step, the hard-way count.
- **If you wrote 36:** That is 6×6. i stops at 5, since $i \le n/2 = 5$.

Takeaway. Throwing work away gives a lower bound for free. The kept steps all really run, since $j \ge n/2 \ge i$ there.

3.3

Answer: **C**

- **A** × It is about half. The argument needs "at least", not "most".
- **B** × It is $n^2/4$. The ¼ is a constant, and Ω does not care about constants.
- **C** ✓ A part is never more than the whole. About $n^2/4$ is $\Omega(n^2)$, since the constant does not matter.
- **D** × Counting a part of the work can only show the work is at least something: a floor.

Takeaway. Upper bounds: round the count up. Lower bounds: throw work away. Neither needs the exact count.

3.4

Answer: **C**

- **A** × n^2 comes from bounding every term by the largest, n. The smallest term, 1, gives n.
- **B** × That is the exact sum. Bounding the terms only ever gives an inequality.
- **C** ✓ Each of the n terms is at least 1. True, and much too weak, because most of the terms are far bigger than 1.

- **D** × It is valid: n terms, each at least 1, add up to at least n. It is just weak here.

Takeaway. CLRS's fix is the easy way: split the sum at n/2, throw the bottom half away, and bound each of the n/2 terms in the top half by n/2. That gives $(n/2)^2$ for even n, which is $\Omega(n^2)$. On the grid it keeps the top n/2 rows, n/2 cells of each: the square, one column narrower.

3.5

Answer: **B**

- **A** × It is: the exact count $n(n + 1)/2$ is at least $n^2/2$.
- **B** ✓ For even n the square has $(n/2)(n/2 + 1) \geq (n/2)^2$ steps. For odd n it has $\lfloor n/2 \rfloor(\lfloor n/2 \rfloor + 1)$, a quarter short of $(n/2)^2$.
- **C** × $i \in \{1, 2\}$ and $j \in \{3, 4, 5\}$ is 6 pairs. The page's grid counts them.
- **D** × Ω needs a floor for all large n, odd included, and $((n - 1)/2)^2$ is one.

Takeaway. A lower bound only has to be true, not tight. Loosen it until it is obviously true, and move on.

3.6

Answer: **Times = 56**

For each i the inner two loops run $1 + 2 + \ldots + i = i(i + 1)/2$ times. Adding those for i = 1 to 6 gives $1 + 3 + 6 + 10 + 15 + 21 = 56$. In general $n(n + 1)(n + 2)/6$, which is $\Theta(n^3)$.

- **If you wrote 216:** That is n^3, as if every loop ran all the way to n.
- **If you wrote 21:** That is the count for i = 6 alone, not the total.
- **If you wrote 36:** That is n^2, one loop too few.

Takeaway. Three nested loops, each bounded by the one outside it: a tetrahedron instead of a triangle, still $\Theta(n^3)$.

3.7

Answer: **D**

- **A** × Only if g is also O(f). Try $f = n$, $g = n^2$.
- **B** × $f + g \geq g$, since $f \geq 0$. The floor is free.
- **C** × The argument uses only $f \geq 0$ and $f \leq cg$.
- **D** ✓ $g \leq f + g$ for the floor, and $f + g \leq (c + 1)g$ for the ceiling.

Takeaway. This is the rule behind dropping lower-order terms: $n^2 + 42n + n \log n$ is $\Theta(n^2)$.

4.1

True: **A, B, C, G** · False: D, E, F

- **A** ✓ It is $O(n^2)$, since $\log n \leq n$.
- **B** ✓ $O(n^2)$. A small constant does not matter.
- **C** ✓ $O(n^2)$.
- **D** × It outgrows every n^d, however large d is.
- **E** × Faster still than 2^n.
- **F** × Faster than any c^n.
- **G** ✓ Polynomial, with d = 100. Impractical, but polynomial.

Takeaway. Polynomial means some fixed degree works. It says nothing about how big that degree is.

4.2

True: **A, B, C** · False: D

- **A** ✓ Slide 17, first reason.
- **B** ✓ Slide 17, second reason. Stable matching: n! by brute force, n^2 by propose-and-reject.
- **C** ✓ Slide 17, third reason.
- **D** × Not so: n^{100} is polynomial. The claim is about what usually happens, not a guarantee.

Takeaway. A definition is useful when it matches practice and when it can be shown false. Polynomial time manages both.

4.3

Answer: **Matchings = 3628800**

The first student has 10 choices, the next 9, and so on: 10! = 3,628,800. Propose-and-reject makes at most $n^2 = 100$ proposals.

- **If you wrote 100:** That is n^2, the bound on proposals. Brute force is far worse.
- **If you wrote 1024:** That is 2^{10}. n! grows faster still.
- **If you wrote 55:** That is 10 + 9 + … + 1. The choices multiply, they do not add.

Takeaway. Ω(n!) against $O(n^2)$ is the gap slide 14 points at: "we must have done something clever".

4.4

Answer: **n = 12**

12! = 479,001,600 steps, under a second; 13! = 6,227,020,800, over 6 seconds. In the same second an n^2 algorithm reaches n = 31,622.

- **If you wrote 13:** 13! is 6,227,020,800 steps: over 6 seconds.
- **If you wrote 29:** That is the answer for 2^n, which fits n = 29 into the same second. n! grows faster still.
- **If you wrote 10:** 10! takes 3.6 ms, so a little more fits.

Takeaway. A machine a thousand times faster reaches only n = 14 for n!, while for n^2 it goes from 31,622 to 1,000,000. A faster computer helps a polynomial algorithm a lot and an exponential one hardly at all.

4.5

Answer: **C**

- **A** × Where a function starts does not decide Big-O. Every exponential passes every polynomial.
- **B** × Once 2^n is ahead, each step doubles it and multiplies n^{10} by less than 2.
- **C** ✓ They cross just below n = 59. From there on, 2^n is ahead for good.
- **D** × $2^n/n^{10}$ grows without bound, so no constant ceiling fits.

Takeaway. Asymptotic means eventually. The crossing point can be late; it cannot be missing.

4.6

Answer: **B**

- **A** × $2^{2n} = 4^n$, and $4^n/2^n = 2^n$ grows without bound. No constant covers it.
- **B** ✓ $2^{n+1} = 2 \cdot 2^n$, so c = 2 works from n = 0. $2^{2n} = 2^n \cdot 2^n$, and the extra factor 2^n is not a constant.

- **C** × Adding 1 to the exponent multiplies by 2, a constant. Doubling the exponent is what changes the class.
- **D** × The other way round.

Takeaway. Adding a constant to an exponent multiplies by a constant; multiplying the exponent changes the base. So 2^{n+1} is $\Theta(2^n)$, and 4^n outgrows both, as 3^n outgrows 2^n on slide 16.

4.7

Answer: **B**

- **A** × That is $n^3 \cdot n^3$. The outer loop is n^2, so it is $n^2 \cdot n^3$.
- **B** ✓ n^2 calls of $O(n^3)$ each is $O(n^5)$. Polynomials multiplied are polynomials.
- **C** × Nothing here is exponential. Multiplying polynomials never leaves them.
- **D** × That applies to terms added together. Nested work multiplies.

Takeaway. Polynomials are closed under adding, multiplying and composing. That is another reason the definition works, and it is the third of the reasons CLRS §34.1 gives for calling polynomial time tractable.

5.1

Answer: **Times = 56**

Each run is one set of three points with $i < j < k$, and each set is visited once, in increasing order. That is the number of ways to choose 3 of 8: $8 \cdot 7 \cdot 6/6 = 56$. In general $n(n-1)(n-2)/6$.

- **If you wrote 512:** That is n^3, as if every loop ran from 1 to n.
- **If you wrote 336:** That counts ordered triples. The loops only visit $i < j < k$, one order of each set of three.
- **If you wrote 28:** That is the number of (i, j) pairs, one loop too few.

Takeaway. When each loop starts just past the one outside it, the count is "choose 3": about $n^3/6$, and $\Theta(n^3)$.

5.2

Answer: **Times = 91**

Cell (i, j) runs j times. Column j is reached by the j rows i = 1 to j, so it contributes $j \cdot j$. Adding: $1 + 4 + 9 + 16 + 25 + 36 = 91$, which is $n(n+1)(2n+1)/6$ at $n = 6$.

- **If you wrote 216:** That is n^3, the ceiling from letting every loop run to n.
- **If you wrote 21:** That counts the (i, j) cells, $n(n+1)/2$, and forgets the k loop.
- **If you wrote 126:** That is as if j started at 1 in every row. It starts at i, so row i misses $1 + \ldots + (i-1)$.

Takeaway. Summing by columns instead of rows turned an awkward double sum into $1^2 + 2^2 + \ldots + n^2$. Pick the order that makes each term simple.

5.3

Answer: **Triples kept = 27**

$i \in \{1, 2, 3\}$, $j \in \{4, 5, 6\}$, $k \in \{7, 8, 9\}$: $3 \times 3 \times 3 = 27$. Every one has $i < j < k$, so every one really runs.

- **If you wrote 84:** That is every triple the program checks, the exact count.
- **If you wrote 729:** That is n^3, the ceiling.
- **If you wrote 9:** That is the 3×3 cells of (i, j), before multiplying by the 3 values of k.

Takeaway. A box is a product of three band widths. Each is about n/3, so the box is about $n^3/27$, which is $\Omega(n^3)$.

5.4

Answer: **C**

- **A** × The box has triples with $j \le i$, such as $i = j = 1$, and the loops never reach those. Its size is not a floor on the work.
- **B** × Most of those triples never run ($i = j = k = 1$, say), and counting the ones that do is the same problem again at n/3.
- **C** ✓ The bands are in order, so $i < j < k$ for every triple in the box, and every one runs. About $(n/3)^3$ of them.
- **D** × That is the ceiling. Most of those triples never run, so it bounds the work above, not below.

Takeaway. The box has to sit entirely inside the work that really happens, and each of its sides has to be a constant fraction of n. Both, or it proves nothing.

5.5

True: **A, B, C** · False: D, E, F

- **A** ✓ The box gives at least about $n^3/8$, and letting every loop run to n gives at most n^3.
- **B** ✓ A ceiling can be as high as you like.
- **C** ✓ A floor can be as low as you like.
- **D** × The count is about $n^3/3$, which passes $c \cdot n^2$ for every c. Three loops over ranges that grow with n.
- **E** × The ceiling half fails, as for $O(n^2)$.
- **F** × The count is at most n^3, and n^3 is not $\Omega(n^4)$.

Takeaway. Only $\Theta(n^3)$ is tight. When an exam asks for "the" running time, it means the Θ, with a floor argument and a ceiling argument both shown.

5.6

Any constants and n_0 that really hold are right. One that does: **$c_1 = 1/27$, $c_2 = 1/6$, $n_0 = 3$** (checked exactly at every whole n from n_0 on).

$c_2 = 1/6$: dropping the − 1 and − 2 only makes each factor bigger, so the count is at most $n^3/6$. $c_1 = 1/27$ is the box of thirds, and it first holds for good at n = 3. So $c_1 = 1/27$, $c_2 = 1/6$, $n_0 = 3$.

Takeaway. The ceiling is "round every factor up", the floor is "keep a box". The constants come straight out of the two arguments.

6.1

Answer: **Multiplications = 55**

x^k takes k multiplications from scratch, so $1 + 2 + \ldots + 10 = 10 \cdot 11/2 = 55$.

- **If you wrote 100:** That is n^2, as if every power took n multiplications.
- **If you wrote 10:** That is the version that reuses P[k − 1]: one multiplication per entry.
- **If you wrote 45:** That is $0 + 1 + \ldots + 9$. The t loop runs k times, not k − 1.

Takeaway. The same triangle as sum-product: $\Theta(n^2)$ for a table of n numbers.

6.2

Answer: **Times as many = 100**

From scratch: $199 \cdot 200/2 = 19900$. Reusing: 199. The ratio is $(n + 1)/2 = 100$.

- **If you wrote 199:** That is the reusing count itself, not the ratio.
- **If you wrote 2:** The ratio is (n + 1)/2, which grows with n. It is not a constant.
- **If you wrote 19900:** That is the from-scratch count itself, not the ratio.

Takeaway. A ratio that grows without bound is what "asymptotically faster" means. Twice as fast at every n would be the same Θ.

6.3

Answer: **C**

- **A** × Each is the same p · x. The difference is how many there are.
- **B** × Both write exactly n: P[1] to P[n].
- **C** ✓ Starting from P[k − 1] instead of from 1 saves k − 1 multiplications for every k.
- **D** × Not always: Print2 is two nested loops and Θ(n). Count, do not guess from the shape.

Takeaway. Look for an inner loop whose work for this round is the previous round's work plus a little. Keep the previous result and add the little.

6.4

Answer: **Multiplications = 11**

The loop runs for i = 10 down to 0, which is n + 1 rounds, with one multiplication x · y in each: 11. Computing each term $a_k x^k$ from scratch instead takes 0 + 1 + … + 10 = 55.

- **If you wrote 10:** i runs from n down to 0: that is n + 1 rounds, not n.
- **If you wrote 55:** That is the naive way, each x^k from scratch: the slow power table with a sum on the end.
- **If you wrote 22:** Each round does one multiplication and one addition. Only the first is a multiplication.

Takeaway. Horner never builds a power on its own: each round multiplies everything so far by x once more, so every earlier coefficient gets its next factor of x for free. Θ(n), and since it has to read all n + 1 coefficients, nothing does better than Ω(n).

6.5

Answer: **A**

- **A** ✓ Then no constant c > 0 has g ≥ c·f for all large n, so g is O(f) but not Ω(f).
- **B** × g = n and f = 2n satisfy it, and they are Θ of each other.
- **C** × That is true when g = f as well.
- **D** × That is a constant factor, the one thing Big-O ignores.

Takeaway. To show a new algorithm is strictly better, work out both counts, divide, and show the ratio goes to 0.

6.6

Answer: **B**

- **A** × True, but far from the best floor available.
- **B** ✓ Every algorithm that produces the table pays for the table. So the plain double loop, n^2 steps, is already optimal.
- **C** × No trick writes n^2 entries in fewer than n^2 steps.
- **D** × This one does not. It holds for every algorithm with this output.

Takeaway. An algorithm can never be faster than its output. When your algorithm's running time matches the size of what it writes, it cannot be beaten by more than a constant.

6.7

True: **A, B, C** · False: D, E, F

- **A** ✓ n^2 entries.
- **B** ✓ n^2 entries, whatever the numbers are.
- **C** ✓ n^3 entries, and a floor of n^3 is also a floor of n^2.
- **D** × n entries: the output gives $\Omega(n)$, and n steps suffice.
- **E** × The answer is one word. Reading the input gives $\Omega(n)$, and neither floor is n^2.
- **F** × The output is n numbers: $\Omega(n)$ from the output. Sorting by comparisons does need $\Omega(n \log n)$ comparisons in the worst case (CLRS Theorem 8.1), but the output alone does not show it.

Takeaway. The output floor is free and always true, and sometimes it is the whole story. When it is far below your algorithm, either the algorithm can improve or you need a different argument.